Who does what, and how we run this

PipelineFortress · 5 people · 3 weeks · all of us have day jobs

Companion doc: **PROJECT_PLAN.md** — *what* we build. This one is *who* and *how*.

The constraint that shapes everything

We all work full time. That rules out the usual student-project model of long shared working sessions. So we're building for **async by default**:

- **Two 30-minute syncs a week.** Tuesday to unblock, Saturday to demo and plan. A third only if something is on fire.
- Everything else is written, in the channel, whenever you're free.
- **Nobody should ever be blocked waiting for a meeting.** That's a design requirement, not a hope.

And one more fact worth stating plainly, from the student guide: **a team's grade is the team's grade.** There's no individual mark to fall back on. One unfinished workstream drags all five of us down — which means our biggest risk is integration, not difficulty.

How we make async actually work

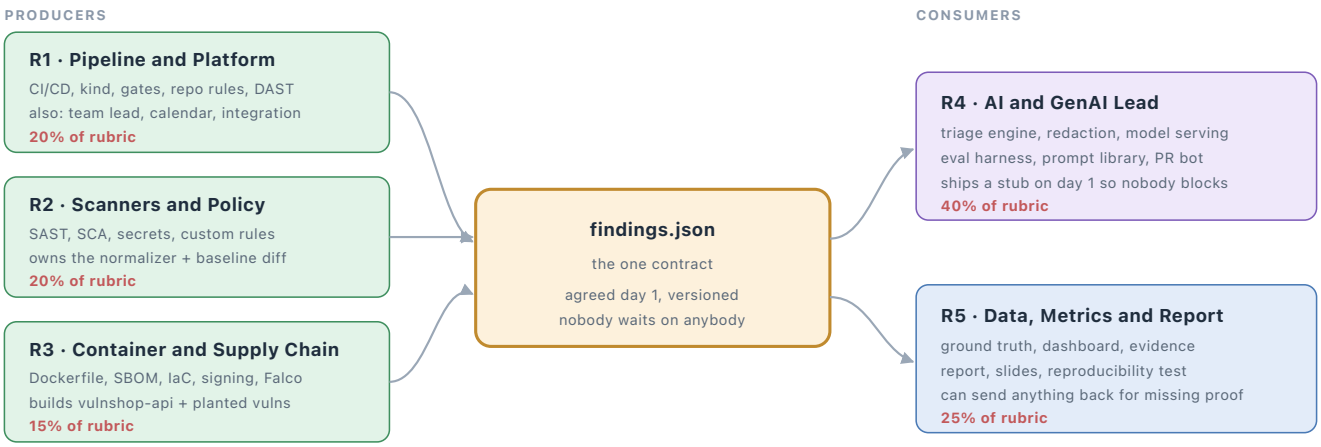
Three mechanisms, and they're the whole trick:

1. One contract, agreed on day 1. The unified finding schema. Once that JSON shape is frozen, all five of us build against it in parallel without talking to each other.

2. Stub first. Anything someone else depends on ships as a stub with the real interface *before* the real thing exists. R4 publishes `stub_triage.py` emitting plausible fake output in week 1, so R1 and R5 can build the PR-comment path and the dashboard against it immediately. **This single move removes the biggest serialisation risk in the project.**

3. Write it down or it didn't happen. Decisions in ADRs, status in the channel, progress on the board. If a decision only exists in someone's head, it doesn't exist.

The five roles



Five vertical slices, one interface. This is what makes async work possible.

Everyone owns a **vertical slice with a visible deliverable** — something you can finish alone on a weeknight without needing anyone else online.

R1 · Pipeline and Platform *(also team lead)*

> *One push runs everything, and everyone can see why it went red.*

Workflows, job graph, caching. kind cluster and staging deploy. **Gate design** — thresholds, blocking vs warning, exception handling. Repo governance: branch protection, `CODEOWNERS`, PR template, required checks. Pipeline hardening: SHA-pin every Action, least-privilege `permissions:` per job, no secrets in the AI job. `make` targets. `act` set up in week 1 so everyone can run workflows locally.

As team lead: the board, the two ceremonies, the risk register, the contribution ledger, and comms with the program coordinator.

Red CI is always R1's P0 — it blocks four other people.

R2 · Scanners and Policy

> *Every tool speaks the same JSON, and the gate only shouts about things that are new.*

Semgrep (+ bandit/gosec on our service), custom rules for our planted vuln classes. Trivy SCA, with **OSV-Scanner and Grype as cross-checks** — that's where the detector-disagreement metric comes from. gitleaks config + a pre-commit hook for the whole team.

Owns the normalizer — every tool's SARIF/JSON into the unified schema. R2 is the custodian of the contract: it gets versioned, and a breaking change after week 1 needs an ADR and a channel heads-up.

Also: dedupe keys, `security-baseline.json`, baseline-diff logic, and the severity-normalisation map (a Trivy HIGH is not a Semgrep ERROR — document the mapping).

R3 · Container, IaC and Supply Chain

> *The image is small, unprivileged, signed, and has a bill of materials.*

Multi-stage hardened Dockerfile: non-root, base pinned **by digest**, no `latest`, read-only rootfs, dropped capabilities. hadolint + Dockle + Trivy image scan. **Syft SBOM in both CycloneDX and SPDX.** Checkov + custom checks, OPA/Rego policies, deliberately-bad manifests for the IaC stage to catch. kube-bench evidence. Stretch: cosign, Kyverno admission, Falco + SIEM stub.

And `vulnshop-api` + `ground_truth/planted.yaml` — the service with planted vulnerabilities.

Why R3 and not R4: the planted vulns must be built by someone *not* on the AI eval team, so they're genuinely unseen by whoever tunes the prompts. That separation is what makes our held-out evaluation honest — **it's a deliberate methodological choice and it goes in the report.**

R4 · AI and GenAI Lead

> *The model is a component. The safety is in the plumbing.*

`llm_client.py` (OpenAI-compatible over Ollama/vLLM). `models.lock.yaml` with pinned digests. **Redaction L1–L4 + canary suite + egress guard.** Clustering. Versioned prompt library. Structured output via JSON Schema + grammar. **Grounding validator.** Prompt-injection corpus and defences. The PR bot. Eval harness, bake-off, ablations. Report sections 7–9 and `AI_DISCLOSURE.md`.

Three hard rules for this role:

1. **Ship the stub in week 1.** Before the real triage exists.
2. **Never the critical path for infrastructure.** If the model isn't ready, the pipeline still runs and emits findings. Degrade, don't block.
3. **Abstain loudly.** `needs_human_review` is a feature. Ship it early so everyone gets used to seeing it.

And delegate. R4 owns 40% of the rubric but should not write 40% of the code:

- Ground-truth labelling → all five label, R5 coordinates
- The 4 AI PRs → R4 generates, **R1/R2/R3 review and write the review notes.** Their names go on the graded artefact, and it forces them to actually engage with the model's output
- Prompt tone and register → pair with R5

R5 · Data, Metrics, QA and Report

> *If it isn't measured and written down, it didn't happen.*

Ground truth: sampling plan, two-labeller protocol, adjudication, **Cohen's κ**, the time-based split, freezing it. Prometheus + **Grafana dashboard**. The metrics catalogue — and keeping vanity metrics out. **Evidence trail**: screenshot standard, naming, index. QA: the day-19 clean-machine reproducibility run, and cross-checking that every claim in the report has an artefact behind it.

The report — owns structure, editing, the Pandoc build. Does *not* write it alone: each role drafts its own sections, R5 integrates and enforces one voice. Plus the deck, the demo script, and both rehearsals.

R5 has standing authority to send any deliverable back for missing evidence — **including R4's.**

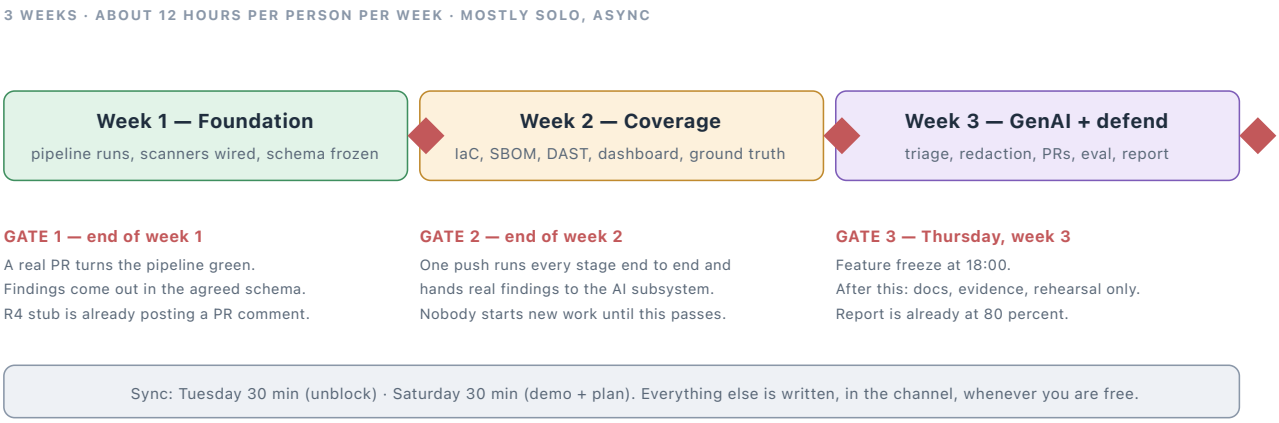
Who's accountable for what

owns = answerable for it landing (exactly one per row — if two people think they own it, nobody does) · **does** = hands on the work · **rev** = reviews or gets consulted before it's final

Deliverable	R1	R2	R3	R4	R5
Pipeline YAML + five stages + gates	**owns**	does	does		
Unified schema + normalizer	rev	**owns**	rev	rev	
Baseline diffing + exceptions	rev	**owns**			
`vulnshop-api` + planted vulns		rev	**owns**		rev
Hardened image + SBOM			**owns**		
Policy-as-code	rev	rev	**owns**		
DAST	**owns**	rev			
LLM triage tool		rev		**owns**	rev
Redaction layer + verification				**owns**	rev
Prompt library				**owns**	
Grounding validator		rev		**owns**	
4 AI-generated PRs	rev	rev	rev	**owns**	
Human review notes on those PRs	does	does	does	rev	**owns**
Ground truth + κ + time split	rev	does	does	rev	**owns**
Eval harness, bake-off, ablations				**owns**	rev
Metrics dashboard	rev			rev	**owns**
Threat model (app / pipeline)	does	does	does	rev	**owns**
Maturity self-assessment	rev	rev	rev	rev	**owns**
Final report	does	does	does	does	**owns**
Deck + demo	does	does	does	does	**owns**
Evidence trail + teardown	does	does	does	does	**owns**

Read the "Human review notes" row as the pattern to copy: **R4 generates the AI PRs, but R1/R2/R3 write the review notes and R5 answers for them existing.** Ownership and the work are deliberately split there.

The calendar



Syncs — two a week, 30 minutes, cameras optional

Tuesday, 30 min — unblock. Not status theatre. One question: *what will stop you shipping by the weekend?* Blockers get assigned in the meeting. If nobody's blocked, it ends in 10 minutes and that's a good outcome.

Saturday, 30 min — demo and plan. Each person shows something working (screen share, 3 min). Set next week's Musts. Update `WEEKLOG.md`. One person does a 5-minute **teach-back** on something they learned — rotate, and record it, because it's also defense practice.

A third sync only when something is on fire. Anyone can call it; keep it to 15 minutes.

Async standup: three lines in the channel whenever you sit down to work. Done / next / blocked. Not daily-at-9am — whenever you actually work.

Realistic hours

About **12 hours a week each**. On top of a job that's roughly **2 hours on four weeknights plus a 4-hour weekend block.** It's a real commitment — block it in your calendar in week 1. The guide's data point: people who try to cram it into week 3 always produce weaker work.

Sunday is off. The guide says so explicitly and it's right.

Week-by-week, per person

Week 1 (~12h each)

R1	repo + protections + board, build/test workflow, kind, deploy Juice Shop, `act`, `make bootstrap`
R2	Semgrep + gitleaks + Trivy wired · **schema v1 agreed with R4 by Tuesday** · normalizer v1 · pre-commit hook
R3	Juice Shop fork pinned · `vulnshop-api` skeleton + **first 6 planted vulns** · base Dockerfile · bad IaC manifests
R4	Ollama + 2 models running · `llm_client.py` · **`stub_triage.py` by Tuesday** ⭐ · redaction L1+L2 spike · prompt v1
R5	report skeleton + notes · evidence convention · `WEEKLOG.md` · ground-truth sampling plan · metrics catalogue

→ **Gate 1:** a real PR turns the pipeline green, findings come out in the agreed schema, the stub posts a comment.

Week 2 (~14h each — the heaviest week)

R1	ZAP against staging · gate thresholds · job parallelism · Action SHA pinning + least-privilege permissions
R2	Checkov · OSV + Grype cross-check · **baseline diffing** · custom rules for planted classes · severity map
R3	hardened multi-stage Dockerfile · Trivy image · **Syft SBOM ×2 formats** · OPA policies · planted vulns to 12–18
R4	clustering + embeddings · enrichment (EPSS/KEV/offline OSV) · redaction L3+L4 + canary suite · triage v1 on real findings
R5	**labelling coordination — all five label ~30 each** · κ computed · split frozen · Grafana v1 · report §§1–6

→ **Gate 2 (end of week 2, non-negotiable):** one push runs every stage end to end and hands real findings to the AI subsystem. *Nobody starts new work until this passes.* This is the failure the student guide explicitly warns about, so we make it a hard stop rather than a hope.

Week 3 (~12h each)

R1	pipeline perf + before/after timings · deterministic `make demo` · **record the fallback video** · teardown
R2	gate tuning from real data · false-fail analysis · detector overlap · **review 1–2 AI PRs + write notes**
R3	Falco + SIEM stub · cosign/Kyverno if green · **review 1–2 AI PRs + write notes** · report §6
R4	grounding validator · injection demo · **4 AI PRs** · held-out eval · **bake-off + ablations** · report §§7–9
R5	dashboard final · all metrics · **report integration pass** · slides · **clean-machine reproducibility test** · rehearsals

→ **Gate 3, Thursday 18:00: feature freeze.** After that: docs, evidence, rehearsal only.

> **Watch the week-3 load on R4 and R5** — it's genuinely heavier. That's why R1/R2/R3 take the PR reviews and draft their own report sections. Check the balance out loud at the Tuesday sync. An overloaded R4/R5 in the final week is the most common way a project like this ends badly.

Working agreements

Git. `main` protected, no direct pushes. Branches `r4/redaction-canary`. Conventional commits. **Commit daily even if it's small** — working code Friday beats perfect code Sunday.

Reviews. Fixed pairs so nothing queues on one person: **R1↔R2 · R3↔R4 · R5 reviews anything**, and everyone reviews R5's report sections. **One review, 24-hour turnaround.** A PR open longer than a day is a blocker — say so in the channel. Keep PRs small; a 600-line PR gets rubber-stamped, which is how bad evidence gets in.

Decisions. Reversible → whoever owns it just decides and announces it. Irreversible or cross-cutting → 24-hour async discussion → **one-page ADR**. Still split? R1 decides, the ADR records the dissent. **No re-litigating settled decisions without new information** — that's the second-biggest time sink after silos.

Board. GitHub Projects. Every issue gets an owner, a tier label (`must` / `should` / `could`), and a rubric label so we can see at a glance where the marks are concentrated. **WIP limit 2 per person.**

Evidence, captured the moment it happens. You cannot recreate a week-1 baseline screenshot in week 3. `artifacts/evidence/<area>/<date>-<desc>.<ext>` plus one line in `INDEX.md` — what it shows, commit SHA,
who captured it.

AI disclosure as we go. One line per use: date, tool, what for, how verified. Includes AI that helps us write code and docs. Doing this honestly is a credibility *gain*. Teams that under-disclose and get asked about it at defense lose more than they saved.

Safety. Anything possibly out of scope or unsafe: stop and escalate the same day. No blame for escalating, real blame for hiding. Includes an accidentally committed secret — rotate, document, disclose.

Cross-training, because the defense is individual

The grade is shared but the panel will ask *you* about someone else's component. Assign **defense buddies** in week 2 — each person can explain one area that isn't theirs, without the owner present:

You own	You must also be able to defend
R1 Pipeline	the AI triage data path and the deterministic/model boundary
R2 Scanners	the redaction layer — four layers, and why canaries matter
R3 Container	the evaluation methodology — ground truth, κ, time split
R4 AI	container and supply chain — SBOM, signing, why digest pinning
R5 Metrics	the pipeline gates — baseline diffing and why it matters

Mechanisms: Saturday teach-backs · **one cross-area PR per person** (tracked) · a Q&A drill at the final Saturday where each person answers three questions on their buddy area.

Contribution ledger

R1 updates `docs/CONTRIBUTIONS.md` weekly. Three purposes: a fair per-member statement for the report appendix, early detection of imbalance, and evidence if anyone's contribution gets questioned.

Week	Member	Merged PRs	Hours	Key deliverables	Cross-area PR	Blocked on

Five minutes at the Saturday sync, out loud. **If someone is drifting it surfaces in week 1 or 2 when it's fixable, not week 3 when it isn't.** Escalate to faculty early if needed — waiting is what turns a workload problem into a grade problem.

Escalation

Situation	What to do
Technical blocker	Channel first, tag the owner. Still stuck after 24h → cohort channel, then office hours
"Is this in scope?"	R1 asks the coordinator. Don't guess — clarifying in week 1 is cheaper than reworking in week 3
Ethics or safety	**Stop immediately**, tell R1 and the program team the same day
Disagreement on approach	24h async → ADR → R1 decides, dissent recorded
Someone overloaded or drifting	Raise it Tuesday, rebalance. R1 → faculty in week 1 or 2, not week 3
Workload or wellbeing	Say it early. The guide explicitly invites this

---

Final checklist

R5 is accountable for all of it being ticked.

- ☐ Every deliverable in brief §6 is present — R5
- ☐ Repo private, program team added — R1
- ☐ Report has the AI-disclosure section — R5
- ☐ Every factual claim traces to an evidence ID — R5
- ☐ **Artifact tested by someone other than its author** *(the day-19 clean-machine run)* — R5
- ☐ Slides 10–15, rehearsed twice — R5
- ☐ Honest limitations section — R5 + R4
- ☐ **Lab torn down, no rogue infrastructure** *(screenshot it)* — R1
- ☐ Responsible AI acknowledgement signed by all 5 **before week 2** — R1
- ☐ `WEEKLOG.md` has three entries — R5
- ☐ `make all` runs clean from a fresh clone in under an hour — R1 + R5
- ☐ 4 AI PRs with review notes, **including one rejected** — R4
- ☐ Bake-off + ablations in the report — R4
- ☐ Canary suite passing, egress log captured — R4
- ☐ Contribution ledger complete — R1
- ☐ Fallback demo video recorded — R1
- ☐ `docs/LICENSES.md` done, **including the model licence** — R3

Name sheet — fill this in at the kickoff and commit it

Role	Name	What they're strong at	Learning goal for these 3 weeks	Defense buddy area
R1 Pipeline *(lead)*				AI triage flow
R2 Scanners				Redaction layer
R3 Container / supply chain				Evaluation methodology
R4 AI/GenAI	*(you)*			Container + supply chain
R5 Metrics / report				Pipeline gates

If you're unsure how to assign: put the most reliable *finisher* on **R5** — the report and the evidence are where marks quietly leak away. Most infra-comfortable person on **R1**. Give **R3** to whoever most wants to learn containers; it's the most self-contained slice and `vulnshop-api` is a genuinely fun build, which also makes it the kindest landing spot for the least experienced member. **R2** suits someone who likes detail and pattern-matching.

R1 keeps this current. If reality diverges from the doc, fix the doc.